一、修改用户名事件流

```
@startuml
start

:打开瑞幸咖啡小程序;
:进入我的页面;
:点击个人头像;
:进入个人资料页面;
:选择修改用户名;
:输入新的用户名;
:点击保存;

:系统验证用户名;

if (验证是否通过?) then (是)
   :更新用户资料;
   :保存到数据库;
   :提示修改成功;
else (否)
   :提示重新输入;
   :重新输入用户名;
endif

stop
@enduml
```

1. 用户打开微信，进入瑞幸咖啡小程序。
2. 系统显示小程序首页。
3. 用户点击右下角的“我的”，进入个人中心。
4. 用户点击头像，进入个人资料页面。
5. 系统显示用户当前的个人资料。
6. 用户选择用户名修改功能。
7. 用户输入新的用户名。
8. 用户点击保存按钮，提交修改信息。
9. 后台系统接收信息并进行验证。
10. 验证通过后，系统将新的用户名保存到数据库。
11. 系统提示修改成功。
12. 用户返回个人资料页面，查看修改后的用户名。

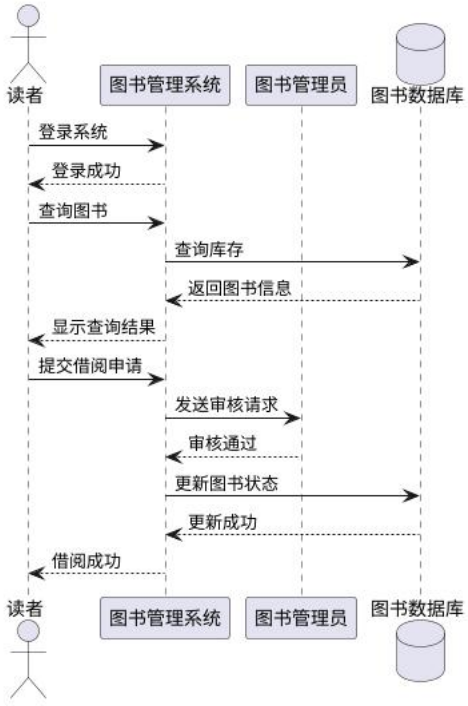
二. 1.顺序图

```
@startuml
actor 读者
participant 图书管理系统
participant 图书管理员
database 图书数据库

读者 -> 图书管理系统: 登录系统
图书管理系统 --> 读者: 登录成功
读者 -> 图书管理系统: 查询图书
图书管理系统 -> 图书数据库: 查询库存
图书数据库 --> 图书管理系统: 返回图书信息
图书管理系统 --> 读者: 显示查询结果

读者 -> 图书管理系统: 提交借阅申请
图书管理系统 -> 图书管理员: 发送审核请求
图书管理员 --> 图书管理系统: 审核通过
图书管理系统 -> 图书数据库: 更新图书状态
图书数据库 --> 图书管理系统: 更新成功
图书管理系统 --> 读者: 借阅成功

@enduml
```



2. 用例图

```
@startuml
left to right direction

actor 读者
actor 图书管理员
actor 系统管理员

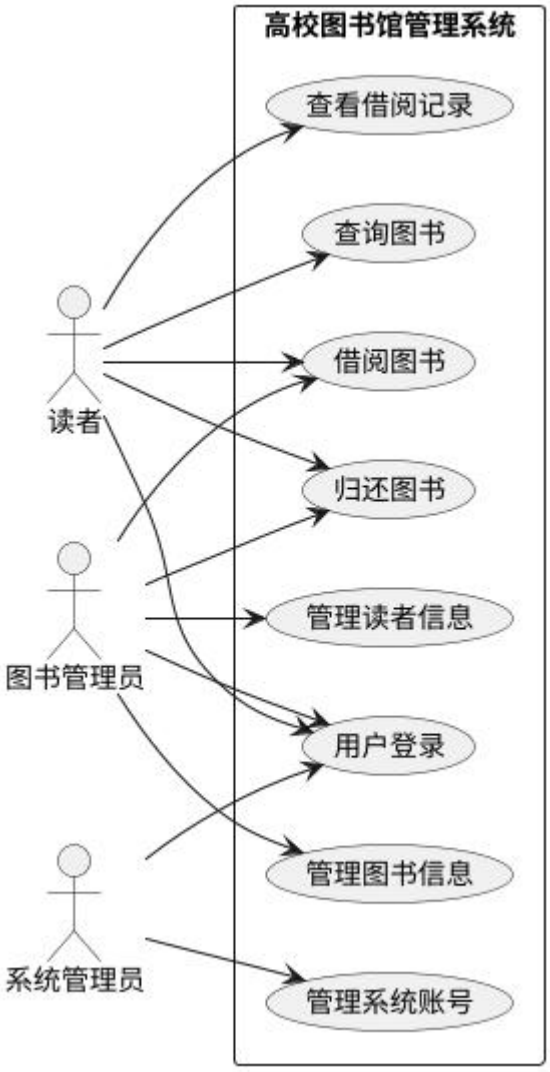
rectangle 高校图书馆管理系统 {
  usecase "用户登录" as A
  usecase "查询图书" as B
  usecase "借阅图书" as C
  usecase "归还图书" as D
  usecase "查看借阅记录" as E
  usecase "管理图书信息" as F
  usecase "管理读者信息" as G
  usecase "管理系统账号" as H
}

读者 --> A
读者 --> B
读者 --> C
读者 --> D
读者 --> E

图书管理员 --> A
图书管理员 --> F
图书管理员 --> G
图书管理员 --> C
图书管理员 --> D

系统管理员 --> A
系统管理员 --> H

@enduml
```


高校图书馆管理系统
查看借阅记录
查询图书
借阅图书
归还图书
管理读者信息
用户登录
管理图书信息
管理系统账号
读者
图书管理员
系统管理员

3. 类图

```
@startuml

class 读者 {
  -读者编号
  -姓名
  -联系方式
  +登录()
  +查询图书()
  +借阅图书()
  +归还图书()
}

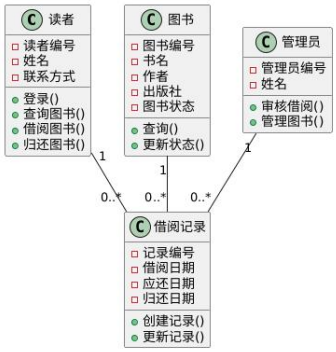
class 图书 {
  -图书编号
  -书名
  -作者
  -出版社
  -图书状态
  +查询()
  +更新状态()
}

class 借阅记录 {
  -记录编号
  -借阅日期
  -应还日期
  -归还日期
  +创建记录()
  +更新记录()
}

class 管理员 {
  -管理员编号
  -姓名
  +审核借阅()
  +管理图书()
}

读者 "1" -- "0..*" 借阅记录
图书 "1" -- "0..*" 借阅记录
管理员 "1" -- "0..*" 借阅记录
```

@enduml



4.对象图

@startuml

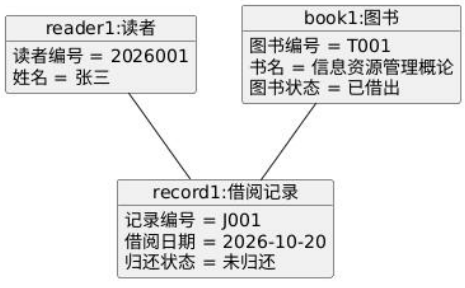
```
object "reader1:读者" as R {
   读者编号 = 2026001
   姓名 = 张三
}

object "book1:图书" as B {
   图书编号 = T001
   书名 = 信息资源管理概论
   图书状态 = 已借出
}

object "record1:借阅记录" as L {
   记录编号 = J001
   借阅日期 = 2026-10-20
   归还状态 = 未归还
}

R -- L
B -- L

@enduml
```



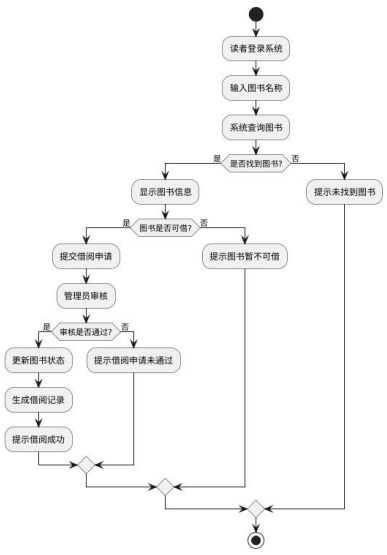
5.活动图

```
@startuml

start
:读者登录系统;
:输入图书名称;
:系统查询图书;

if (是否找到图书?) then (是)
   :显示图书信息;
   if (图书是否可借?) then (是)
      :提交借阅申请;
      :管理员审核;
      if (审核是否通过?) then (是)
         :更新图书状态;
         :生成借阅记录;
         :提示借阅成功;
      else (否)
         :提示借阅申请未通过;
      endif
   else (否)
      :提示图书暂不可借;
   endif
else (否)
   :提示未找到图书;
endif

stop
@enduml
```



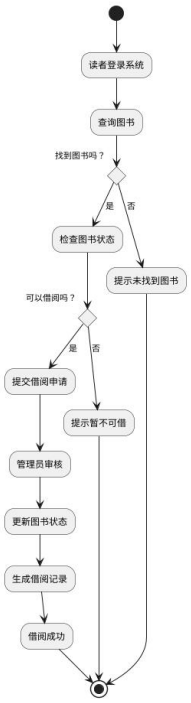
6. 旧版活动图语法

```
@startuml

(*) --> "读者登录系统"
--> "查询图书"

if "找到图书吗？" then
   -->[是] "检查图书状态"
   if "可以借阅吗？" then
      -->[是] "提交借阅申请"
      --> "管理员审核"
      --> "更新图书状态"
      --> "生成借阅记录"
      --> "借阅成功"
      --> (*)
   else
      -->[否] "提示暂不可借"
      --> (*)
   endif
else
   -->[否] "提示未找到图书"
   --> (*)
endif

@enduml
```



7. 组件图

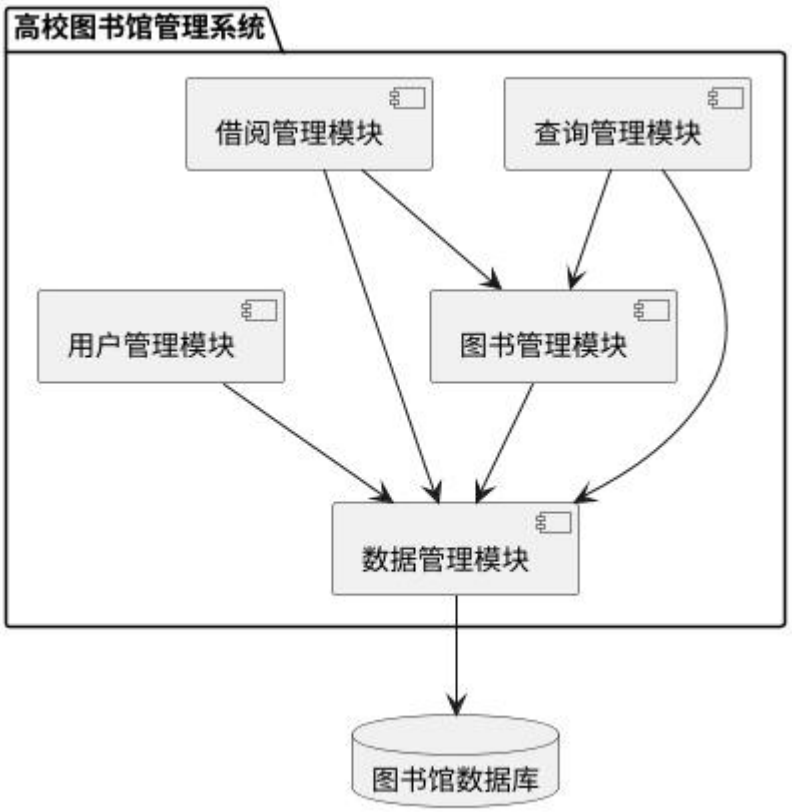
```
@startuml

package "高校图书馆管理系统" {
   [用户管理模块] as A
   [图书管理模块] as B
   [借阅管理模块] as C
   [查询管理模块] as D
   [数据管理模块] as E
}

database "图书馆数据库" as DB

A --> E
B --> E
C --> B
C --> E
D --> B
D --> E
E --> DB

@enduml
```



8. 部署图

```
@startuml

node "读者终端" {
  artifact "浏览器/客户端"
}

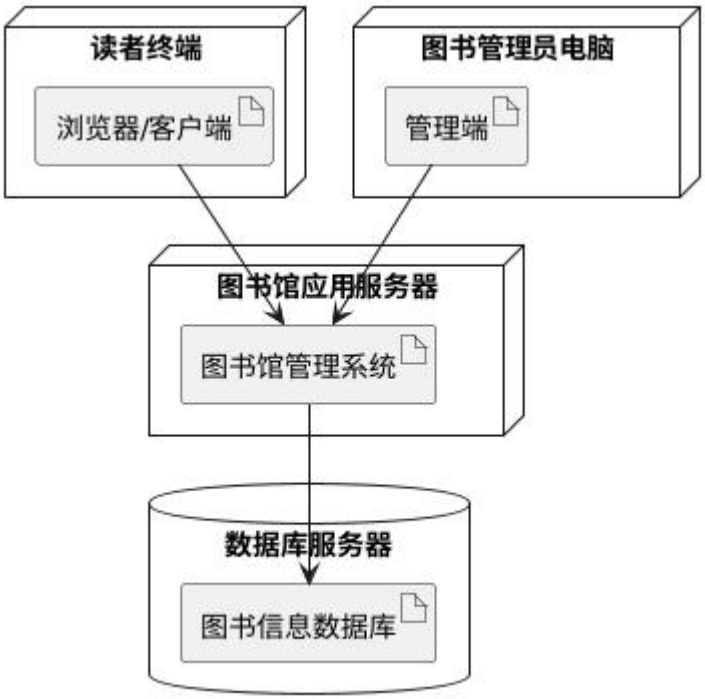
node "图书管理员电脑" {
  artifact "管理端"
}

node "图书馆应用服务器" {
  artifact "图书馆管理系统"
}

database "数据库服务器" {
  artifact "图书信息数据库"
}

"浏览器/客户端" --> "图书馆管理系统"
"管理端" --> "图书馆管理系统"
"图书馆管理系统" --> "图书信息数据库"

@enduml
```



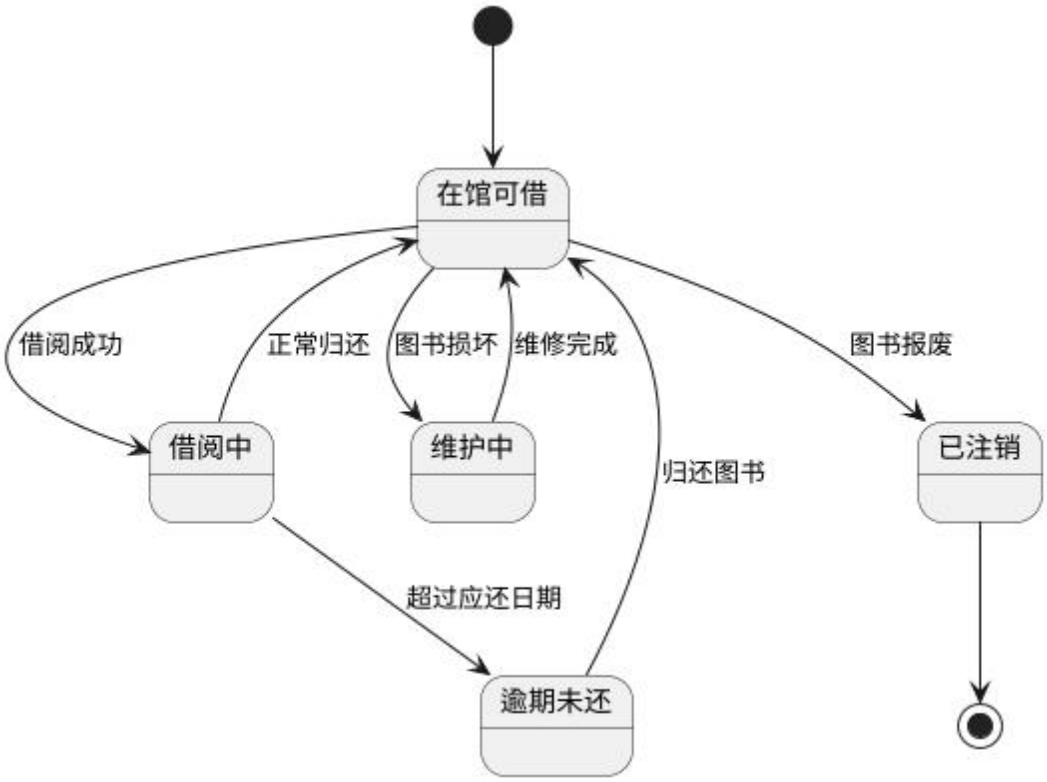
9. 状态图

@startuml

[*] --> 在馆可借

在馆可借 --> 借阅中 : 借阅成功
借阅中 --> 在馆可借 : 正常归还
借阅中 --> 逾期未还 : 超过应还日期
逾期未还 --> 在馆可借 : 归还图书
在馆可借 --> 维护中 : 图书损坏
维护中 --> 在馆可借 : 维修完成
在馆可借 --> 已注销 : 图书报废
已注销 --> [*]

@enduml



10. 定时图

@startuml

robust "读者" as R
concise "图书管理系统" as S
concise "数据库" as D

@0
R is 空闲
S is 等待操作
D is 正常

@10
R is 提交申请
S is 接收申请
D is 正常

@20
R is 等待结果
S is 验证信息
D is 查询数据

@30
R is 等待结果
S is 处理借阅
D is 更新记录

@40
R is 查看结果
S is 返回成功
D is 保存完成

@enduml

